SIH26227: Technical Stack & Library Breakdown

A complete guide for presentation defense.

If judges ask you what technologies were used and WHY they were used, use this document to answer confidently. Our architecture is split into Frontend, Backend, Machine Learning, and Databases.

1. Frontend (User Interface)

React.js

What it is: A JavaScript library for building user interfaces.
Why we used it: It allows us to build the map dashboard, image swipe tool, and search bar as reusable components. It makes the UI highly interactive without refreshing the page.

TypeScript

What it is: A strongly typed version of JavaScript.
Why we used it: It prevents bugs during development by catching errors early (e.g., ensuring coordinates are numbers, not text).

Vite

What it is: A modern, lightning-fast build tool for web apps.
Why we used it: It compiles our React code instantly, making development much faster than older tools like Webpack.

Tailwind CSS

What it is: A utility-first CSS framework.
Why we used it: Instead of writing thousands of lines of custom CSS, we use Tailwind classes (like 'bg-blue-500') to style our app directly inside the HTML, saving massive amounts of time.

2. Backend (Server & APIs)

Python

What it is: The core programming language of our backend.
Why we used it: It is the industry standard for Artificial Intelligence and Machine Learning, allowing seamless integration with our models.

FastAPI

What it is: A high-performance web framework for building APIs in Python.
Why we used it: It is incredibly fast. We used it to create the endpoints (like /api/search) that the React frontend talks to.

Uvicorn

What it is: An ASGI web server implementation for Python.
Why we used it: It actually runs our FastAPI application in the background and handles incoming web traffic.

Pandas

What it is: A powerful data manipulation library in Python.
Why we used it: We used it to filter our satellite metadata. When a user filters by 'Date' or 'Sensor', Pandas instantly filters the massive dataset.

3. Machine Learning & AI

PyTorch

What it is: A premier open-source deep learning framework.
Why we used it: It allows us to run our AI models entirely offline (Air-gapped) on the local CPU/GPU, fulfilling the strict military/defense requirement.

ResNet-18 (Foundation Model Proxy)

What it is: A Convolutional Neural Network (CNN).
Why we used it: It reads satellite images and converts them into 768-Dimensional mathematical vectors (embeddings). This allows us to search by image visually rather than just by text.

DeepLabV3

What it is: A state-of-the-art Deep Image Segmentation model.
Why we used it: We use it to compare the 'before' and 'after' satellite images pixel-by-pixel to detect exact morphological changes (like a lake expanding).

4. Databases & Search

FAISS (Facebook AI Similarity Search)

What it is: A library developed by Facebook for efficient similarity search of dense vectors.
Why we used it: When you upload an image, the AI turns it into a vector. FAISS compares that vector against millions of others in milliseconds to find similar locations.

Scikit-Learn (K-Means)

What it is: A machine learning library.
Why we used it: We used its K-Means algorithm to automatically group (cluster) similar changes together in the 'Cluster Explorer' tool.

SQLite

What it is: A lightweight, file-based relational database.
Why we used it: We used it to store the 'Analyst Feedback' (when a user clicks Approve/Reject) directly on the local hard drive, keeping the app 100% offline.

Key Defense Points for Judges:

• If they ask 'Why not use standard databases like MySQL?': Explain that traditional databases can't do image similarity search. We NEEDED a Vector Database (FAISS) to search images mathematically.

• If they ask 'Why did you use PyTorch instead of calling ChatGPT/OpenAI?': Explain that defense/satellite platforms must be AIR-GAPPED (100% Offline). PyTorch allows the AI to run locally without internet.

• If they ask 'How does it run so fast?': Explain that FAISS indexes vectors in RAM, and FastAPI handles asynchronous web requests.

• If they ask 'How do you prevent false alarms from clouds?': Explain that we mathematically suppress confidence scores for tiles that exhibit high noise variance before sending them to the user.